Contents

DreamCoder List Domain Primitives: A Detailed Analysis

This document provides a comprehensive analysis of the primitives defined in `dreamcoder/domains/list/listPrimi` from the DreamCoder repository (ellisk42/ec).

Table of Contents

Overview

DreamCoder's list domain is one of the core domains used to demonstrate program synthesis. The primitives define a functional programming language for list manipulation, inspired by LISP and lambda calculus.

Key concepts: - All functions are **curried** (multi-argument functions return functions) - Types use **polymorphism** via type variables (`t0`, `t1`, `t2`) - Multiple **primitive sets** exist for different experimental conditions - **Recursion** is explicit via fixed-point combinators (`fix1`, `fix2`)

Primitive Categories and Their Logic

The primitives are organized into categories that reflect their **computational role** rather than arbitrary groupings. Understanding why these categories exist helps clarify the DSL's design.

First-Order vs Higher-Order: The Key Distinction

The most important organizational principle is the **order** of functions:

Category	Order	What it means	Examples
List Operations	First-order	Take/return *data* only	`cons`, `car`, `cdr`, `reverse`
Higher-Order Functions	Higher-order	Take *functions* as arguments	`map`, `filter`, `fold`

Why this matters: Higher-order functions dramatically increase expressiveness. With first-order primitives, you can only manipulate specific data. With higher-order primitives, you can abstract over *behavior itself.*

Consider the type signatures: - `cons : t0 -> list(t0) -> list(t0)` – takes data, returns data - `map : (t0 -> t1) -> list(t0) -> list(t1)` – takes a *function* `(t0 -> t1)` as its first argument

Higher-order functions let you write one abstraction (`map`) that works for infinitely many specific operations (double each element, negate each element, etc.).

Predicates vs Connectives: The Boolean Pipeline

The distinction between "Comparison" and "Boolean" primitives reflects their role in computation:

Category	Role	Type Pattern	Examples
Comparisons (Predicates)	*Produce* booleans from data	`t -> t -> bool`	`eq?`, `gt?`, `is-prime`
Boolean Connectives	*Consume/combine* booleans	`bool -> bool -> bool`	`and`, `or`, `not`
Control Flow	*Use* booleans for branching	`bool -> t0 -> t0 -> t0`	`if`

These form a pipeline:

`Data -> [Predicate] -> Boolean -> [Connective] -> Boolean -> [Control] -> Result`

For example: `(if (and (gt? x 0) (is-prime x)) "yes" "no")` - `gt?` and `is-prime` produce booleans - `and` combines them - `if` uses the result to select a branch

Arithmetic: The Numeric Substrate

Arithmetic primitives provide the numeric operations needed for list indexing, counting, and numeric predicates:

Primitive	Type	Role
`+`, `-`, `*`	`int -> int -> int`	Basic arithmetic
`mod`	`int -> int -> int`	Needed for `is-prime`, cyclic patterns
`negate`	`int -> int`	Sign manipulation
`0..5`	`int`	Constants (see discussion below)

Complete Primitive Reference

Integer Constants

Name	Type	Description
`0`, `1`, `2`, `3`, `4`, `5`	`int`	Literal integers

Why 0-5? The benchmark tasks rarely need larger constants. Theoretically, you only need `0` and a successor function `(+1)`, but this makes programs unnecessarily long. Having constants 0-5 is a pragmatic compromise between minimality and program brevity.

In `McCarthyPrimitives()`, only `0` and `1` are provided – the theoretical minimum.

First-Order List Operations

These manipulate list *data* without taking functions as arguments.

Name	Type Signature	Description
`empty`	`list(t0)`	Empty list `[]`
`singleton`	`t0 -> list(t0)`	Wrap element: `x -> [x]`
`cons`	`t0 -> list(t0) -> list(t0)`	Prepend: `x, [y,z] -> [x,y,z]`
`car`	`list(t0) -> t0`	First element (head): `[x,y,z] -> x`
`cdr`	`list(t0) -> list(t0)`	Rest (tail): `[x,y,z] -> [y,z]`
`empty?`	`list(t0) -> bool`	Test if empty
`++`	`list(t0) -> list(t0) -> list(t0)`	Concatenate lists
`range`	`int -> list(int)`	Generate `[0, 1, ..., n-1]`

Name	Type Signature	Description
`length`	`list(t0) -> int`	List length
`reverse`	`list(t0) -> list(t0)`	Reverse order
`sort`	`list(int) -> list(int)`	Sort integers ascending
`index`	`int -> list(t0) -> t0`	Element at position i
`slice`	`int -> int -> list(t0) -> list(t0)`	Extract sublist `[i:j]`

The McCarthy Core: `empty`, `cons`, `car`, `cdr`, `empty?` are the minimal list primitives from McCarthy's 1959 LISP. Everything else can be built from these (plus recursion).

Higher-Order Functions

These take *functions* as arguments, enabling abstraction over behavior.

Name	Type Signature	Description
`map`	`(t0 -> t1) -> list(t0) -> list(t1)`	Apply f to each element
`mapi`	`(int -> t0 -> t1) -> list(t0) -> list(t1)`	Map with index
`filter`	`(t0 -> bool) -> list(t0) -> list(t0)`	Keep elements satisfying predicate
`fold`	`list(t0) -> t1 -> (t0 -> t1 -> t1) -> t1`	Right fold (process right-to-left)
`reducei`	`(int -> t1 -> t0 -> t1) -> t1 -> list(t0) -> t1`	Reduce with index
`zip`	`list(t0) -> list(t1) -> (t0 -> t1 -> t2) -> list(t2)`	Combine two lists with function
`unfold`	`t0 -> (t0 -> bool) -> (t0 -> t1) -> (t0 -> t0) -> list(t1)`	Generate list from seed
`all`	`(t0 -> bool) -> list(t0) -> bool`	All elements satisfy predicate?
`any`	`(t0 -> bool) -> list(t0) -> bool`	Any element satisfies predicate?

Note: `primitives()` uses `mapi`/`reducei` (indexed versions) instead of plain `map`/`reduce`. This is an interesting design choice – it makes the index available without needing to compute it.

Arithmetic

Name	Type Signature	Description
`+`	`int -> int -> int`	Addition
`-`	`int -> int -> int`	Subtraction
`*`	`int -> int -> int`	Multiplication
`mod`	`int -> int -> int`	Modulo
`negate`	`int -> int`	Negation: `x -> -x`
`sum`	`list(int) -> int`	Sum of list elements

Predicates (Produce Booleans)

Name	Type Signature	Description
`eq?`	`int -> int -> bool`	Equality test
`gt?`	`int -> int -> bool`	Greater-than test
`is-prime`	`int -> bool`	Primality test (hardcoded primes <= 199)
`is-square`	`int -> bool`	Perfect square test

Boolean Connectives (Consume/Combine Booleans)

Name	Type Signature	Description
`true`	`bool`	Boolean constant
`not`	`bool -> bool`	Logical negation
`and`	`bool -> bool -> bool`	Logical AND
`or`	`bool -> bool -> bool`	Logical OR

Control Flow

Name	Type Signature	Description
`if`	`bool -> t0 -> t0 -> t0`	Conditional: `if c then t else f`

Recursion (Fixed-Point Combinators)

Name	Type Signature	Description
`fix1`	`t0 -> ((t0 -> t1) -> t0 -> t1) -> t1`	Y-combinator for unary recursion
`fix2`	`t0 -> t1 -> ((t0 -> t1 -> t2) -> t0 -> t1 -> t2) -> t2`	Y-combinator for binary recursion

Why fixed-point combinators? Pure lambda calculus has no built-in recursion. To define recursive functions, you need a fixed-point combinator. `fix1` lets you write:

```
length = fix1 [] (\self. \xs. if (empty? xs) 0 (+ 1 (self (cdr xs))))
```

The `self` parameter becomes the recursive call. DreamCoder limits recursion depth to 20 to prevent infinite loops during synthesis.

Primitive Sets Explained

DreamCoder defines **multiple primitive sets** for different experimental purposes. Understanding the logic behind each set is crucial.

The Inclusion Logic: How Do They Decide?

The decision of what to include follows these principles:

1. **McCarthy primitives** = Theoretical minimum for Turing-complete list processing
2. **Derived primitives** are included if they are:
   - Expressible with McCarthy primitives (not strictly necessary)
   - Frequently useful across tasks (high reuse value)
   - Unlikely to be rediscovered during search (too complex)
   - Significantly compress the grammar when abstracted

The source code comments reveal this thinking explicitly:

```
# these are achievable with above primitives, but unlikely
Primitive("sum", ...),      # Would be: (reduce (\a \x (+ a x)) 0 $0)
Primitive("reverse", ...),  # Would be: (reduce (\a \x (++ (singleton x) a)) empty $0)
```

So `sum` is included because while you *could* write it as a fold, the system is unlikely to stumble upon that exact combination during enumeration.

1. `McCarthyPrimitives()` – The Theoretical Minimum (1959 LISP)

Purpose: Historical baseline – the minimal foundation for list computation.

Primitives (14 total):

```
Constants:     0, 1
List core:     empty, cons, car, cdr, empty?
Arithmetic:    +, -
Comparison:    eq?, gt?
Control:       if
Recursion:     fix1
```

What's missing: - No `*`, `mod` (can be built from `+` and recursion) - No `map`, `filter`, `fold` (can be built from `cons`/`car`/`cdr` + recursion) - Only constants 0 and 1 (others built via `(+ 1 (+ 1 0))` etc.)

Use case: Ablation – can the system learn everything from the absolute minimum?

---

2. `basePrimitives()` – McCarthy + Domain Helpers

Purpose: Practical foundation for the list benchmark.

Primitives (19 total):

```
Constants:     0, 1, 2, 3, 4, 5
List core:     empty, cons, car, cdr, empty?
Arithmetic:    +, -, *
Comparison:    eq?, gt?
Control:       if
Domain-specific: is-prime, is-square
```

What's added over McCarthy: - Constants 0-5 (practical, shorter programs) - Multiplication (common operation) - `is-prime`, `is-square` (needed for benchmark tasks)

What's still missing: - No higher-order functions (`map`, `filter`, `fold`) - No `range`, `reverse`, `length` - No boolean combinators (`and`, `or`, `not`)

Use case: Test if DreamCoder can *learn* higher-order abstractions from first-order primitives.

---

3. `bootstrapTarget()` – What We Hope to Learn

Purpose: Defines the target abstractions the system should discover through wake-sleep.

Structure: Split into "built-ins" (given) and "learned" (targets):

```
BUILT-INS (given at start):
  Constants:   0, 1
  List core:   empty, cons, car, cdr, empty?
  Arithmetic:  +, -
  Control:     if

LEARNED (what should emerge):
  map      : (t0 -> t1) -> list(t0) -> list(t1)
  fold     : list(t0) -> t1 -> (t0 -> t1 -> t1) -> t1
  unfold   : t0 -> (t0 -> bool) -> (t0 -> t1) -> (t0 -> t0) -> list(t1)
  range    : int -> list(int)
  index    : int -> list(t0) -> t0
  length   : list(t0) -> int
```

The key insight: The "learned" primitives can all be expressed using the "built-ins":

Learned	Equivalent using built-ins
`length`	`(fix1 $0 (\self \xs (if (empty? xs) 0 (+ 1 (self (cdr xs))))))`
`map`	`(fix1 $0 (\self \xs (if (empty? xs) empty (cons (f (car xs)) (self (cdr xs))))))`
`range`	`(fix1 $0 (\self \n (if (eq? n 0) empty (cons (- n 1) (self (- n 1))))))`

But these are complex! The system should discover them through compression.

Use case: Evaluating library learning – does the wake-sleep loop actually discover `map`, `fold`, etc.?

---

4. `bootstrapTarget_extra()` – Bootstrap + Domain Specifics

Purpose: Full capability for the list benchmark after bootstrapping.

Adds to bootstrapTarget:

```
Arithmetic:      *, mod
Comparison:      eq?, gt?
Domain-specific: is-prime, is-square
```

Use case: Running the full benchmark after the system has (hopefully) learned the core abstractions.

---

5. `primitives()` – The Full Feature Set

Purpose: Maximum expressiveness for solving diverse list tasks.

Key differences from other sets: - Uses `mapi`/`reducei` instead of `map`/`reduce` (indexed versions) - No `if` statement! - Includes convenience functions: `sort`, `sum`, `reverse`, `all`, `any`, `slice` - Has `singleton` and `++` for list construction

Primitives (29 total):

```
Constants:      0, 1, 2, 3, 4, 5
List ops:       empty, singleton, range, ++, reverse, sort, index, slice
Higher-order:   mapi, reducei, filter, all, any
Arithmetic:     +, *, negate, mod, sum
Comparison:     eq?, gt?, is-prime, is-square
Boolean:        true, not, and, or
```

Use case: Solving the full list benchmark with rich primitives. Many programs become short.

---

6. `no_length()` – Reviewer-Requested Ablation

Purpose: Measure how important `length` is.

Contents: Same as `bootstrapTarget_extra()` but removes `length`.

Use case: A reviewer asked "what if you didn't have `length`?" This answers that question.

Design Philosophy

The Hierarchy of Primitive Sets

```
McCarthyPrimitives (most minimal -- 14 primitives)
    |
    | add: *, constants 2-5, is-prime, is-square
    v
basePrimitives (practical minimum -- 19 primitives)
    |
    | add: map, fold, unfold, range, index, length
    v
bootstrapTarget (learning evaluation -- 16 primitives)
    |
    | add: *, mod, gt?, eq?, is-prime, is-square
```

```
    v
bootstrapTarget_extra (full capability -- 22 primitives)


primitives (separate branch -- 29 primitives, uses mapi/reducei)
```

Why Multiple Sets?

1. **Ablation studies:** Each set removes capabilities to measure their contribution
2. **Learning evaluation:** Start minimal, measure what abstractions emerge
3. **Historical comparison:** Compare to McCarthy's 1959 LISP foundation
4. **Practical benchmarking:** Rich sets for solving real tasks efficiently

Currying: Why Everything is `t0 -> t1 -> t2` not `(t0, t1) -> t2`

All multi-argument functions are curried. Instead of `add(x, y)`, you have `add(x)(y)`.

Benefits: - Enables partial application: `(+ 1)` is "the function that adds 1" - Simplifies the type system - Natural fit for lambda calculus - Makes higher-order programming elegant

Recursion via Fixed-Point: Why `fix` Instead of Direct Recursion?

DreamCoder uses fixed-point combinators instead of allowing functions to call themselves by name.

Reasons: 1. Keeps the language pure (no named definitions) 2. Makes program complexity measurable (recursion is explicit) 3. Enables depth limiting (prevent infinite loops during synthesis) 4. Theoretically cleaner (closer to pure lambda calculus)

Source Reference

The complete source is available at: https://github.com/ellisk42/ec/blob/master/dreamcoder/domains/list/listPrimi